Methodology of Contrastive Self-Supervised Learning

Contrastive self-supervised learning follows a **fixed pipeline**. The idea is:

Create positive and negative pairs → Encode them → Compare them with a contrastive loss → Learn representations → Use them for downstream tasks.

1. Data Preparation & Pretext Task (Generate Positive/Negative Pairs)

A *pretext task* is used to create signals (pseudo-labels) from the data itself.

How it works

- **Take a sample xxx → call it anchor.**
- **Apply one or more augmentations (color jitter, crop, rotation, jigsaw, temporal shuffle, etc.)**
- **The augmented version x+x^+x+ = positive pair.**
- **All other samples (or their stored embeddings) = negative pairs.**

Why this step exists

It forces the model to learn invariant features (e.g., rotation-invariant, color-invariant, view-invariant).

---

2. Encoder Network

A deep model (commonly ResNet-50) encodes each sample into a vector representation.

- **For an anchor → get embedding qqq**

- **For a positive or negative → get embedding kkk**

Often a projection head (MLP) is added on top of the encoder to map representations into a space where contrastive loss works better.

---

3. Architectural Strategy (How negatives are obtained)

The survey groups contrastive methods into four architectures:

(a) End-to-End Training

- **Big batch size → negatives come from the batch.**
- **Example: SimCLR.**

(b) Memory Bank

- **Store embeddings of past samples.**
- **Retrieve negatives from this bank.**
- **Example: PIRL, NPID.**

(c) Momentum Encoder

- **A second encoder updated slowly (momentum update).**
- **Stores a queue of recent negative embeddings.**
- **Example: MoCo.**

(d) Clustering-Based

- **No explicit negatives. Similar samples grouped via cluster assignments.**
- **Example: SwAV.**

Each method differs only in where the negatives come from or how similarity is enforced.

---

4. Contrastive Loss (Learning Step)

The core is InfoNCE / NCE loss:

$$\mathcal{L} = -\log \frac{\exp(sim(q, k^+)/\tau)}{\exp(sim(q, k^+)/\tau) + \sum_i \exp(sim(q, k_i^-)/\tau)}$$

Where:

- q = anchor embedding
- k^+ = positive
- k_i^- = negatives
- $sim()$ = cosine similarity
- τ = temperature

This loss enforces:

- **Pull together: anchor & positive**
- **Push apart: anchor & negatives**

The model learns meaningful latent space structure without labels.

---

5. Training the Model

Use optimizers like:

- **SGD / SGD + momentum**
- **Adam**

- **LARS (for large batches)**

Training involves:

- **Repeatedly generating augmented views**
- **Passing them through encoders**
- **Minimizing contrastive loss**

---

6. Downstream Task Evaluation (After Training)

Finally, evaluate the learned representations by transferring them to:

- **Image classification**
- **Object detection**
- **Segmentation**
- **Action recognition**
- **NLP tasks (for text-based contrastive models)**

This demonstrates how well the learned features generalize.

Summary (One-Line Methodology)

Generate augmented views → encode them → contrast positives vs negatives → learn representations → transfer to downstream tasks.

What is an Encoder in Contrastive Learning?

In contrastive self-supervised learning, the encoder is the *heart* of the entire methodology.
Its job is simple but extremely important:

> **Take raw input (image/video/text) → convert it into a meaningful vector representation.**

This representation is what the contrastive loss compares.

Think of it as the feature extractor of the entire system.

Why Do We Need an Encoder?

Because contrastive learning does not use labels.
So the only way the model "understands" similarity/difference is through:

- **how it encodes an input**
- **how close or far those encodings are**

If the encoder is weak → everything downstream (classification, detection, clustering) will also be weak.

What Exactly Does the Encoder Do?

1. Maps raw data to latent vectors

Example for images:

- **Input: 224×224 RGB image**
- **Output: a 2048-dimensional vector**

Example for videos:

- **Input: 16 frames**
- **Output: a multi-frame spatiotemporal embedding**

Example for text:

- **Input: a sentence**
- **Output: a sentence embedding**

This vector captures the underlying structure/patterns of the data.

---

2. Learns useful patterns Without Labels

It learns:

- **shapes**
- **textures**
- **context**
- **object boundaries**
- **semantic meaning**
- **motion patterns (for video)**
- **sentence semantics (for text)**

The encoder doesn't know the class “cat” — but it learns that cat images produce similar embeddings.

---

3. Works with Two Streams: Query & Key

Most contrastive methods use two encoders or two paths:

- **Query Encoder (Q):**
  Processes the original image (anchor).

- **Key Encoder (K):**
  Processes augmented images (positive + negatives).

Why two?
Because they help produce *slightly different* views of the same data → crucial for contrastive learning.

Some systems use:

- **two separate encoders trained together (SimCLR)**

- **momentum encoder (MoCo)**

- **shared encoder with clustering (SwAV)**

The goal is always:
differentiate between same-sample views vs other samples.

---

4. The Encoder Outputs Go Into a Projection Head

Most modern systems add an extra MLP called a projection head.

Why?

Because the raw encoder output ("features") is too general.
The projection head compresses them into a space where the contrastive loss works better.

Example:

- **Encoder output = 2048-d**

- **Projection head output = 128-d**

The contrastive comparisons happen in this smaller space.

---

5. Different Encoders for Different Modalities

Images → CNNs

Most common: ResNet-50

Because:

- **deep, stable**
- **good inductive bias**
- **widely used in supervised learning**

Videos → 3D CNNs

Example: 3D-ResNet
It learns from time + space together.

Text → Transformers

Such as:

- **BERT encoder**
- **GPT-style layers**
- **Sentence transformers**

These handle language semantics and context.

Graphs → GNN Encoders

Used for graph-contrastive learning.

---

6. The Encoder is Later Used for Downstream Tasks

After contrastive pretraining:

- **The encoder is frozen or fine-tuned**
- **A classifier head is attached**
- **It performs exceptionally well with *very few* labeled examples**

This is the main advantage of contrastive learning:
learn good representations first → perform well later.

---

7. Why Encoder Choice Matters

A good encoder leads to:

- **better invariance to augmentations**
- **richer feature representations**
- **better generalization**
- **higher downstream accuracy**

Thus, encoder architecture is critical for performance.

In One Sentence:

The encoder is the central neural network that transforms raw input into feature vectors that the contrastive objective uses to learn similarity and dissimilarity without labels.